

[Skip to main content](#)  
[Enable accessibility for low vision](#)  
[Open the accessibility menu](#)



## CodePath Courses

Courses

Quick Links

Quick Actions



Giancarlo Montes

# CYB102 | Intermediate Cybersecurity

Intermediate Cybersecurity Fall 2024 (@ Section 1 | Tuesdays 5PM - 7PM PDT)

Personal Member ID#: **113724**

Need help? Post on our [class slack channel](#) or email us at [support@codepath.org](mailto:support@codepath.org)

- Getting Started
- Course Overview
- Schedule
- VM Access Guide
- AI Usage
- My Report
- Individual Projects
  - Unit 1
  - Unit 2
  - Unit 3**
  - Unit 4
  - Unit 5
  - Unit 6
  - Unit 7

Group Capstone

Groupings  
Career Resources

Pework

Survey  
Overview  
Lab  
Project  
Resources  
Career

# Lab #3: That's Snort of a Lot of Rules

---

## 🚩 MACHINE ACCESS CHECK 🚩

To proceed with this unit, please ensure you have access to a **cyb102 Ubuntu VM** as detailed in the [🔗 VM Access Guide](#).

We recommend you begin with **Azure Labs**.

✅ All options in the [🔗 VM Access Guide](#) will work for this unit.

## Overview

**Snort** is a network intrusion detection system (NIDS) that uses signatures and rule sets to monitor real-time network traffic and detect malicious activities via packet analysis.

Today we will install Snort and write Snort rules that would be able to detect this traffic.

## 🎯 Goals

By the end of this lab you will:

- Employ the pybull-ng testing framework to view network attacks in real-time
- Configure Snort to track network communication
- Write a custom Snort rule to prevent specific network activity

**Note:** There are no stretch goals for this lab.

## Resources

[TryHackMe's Snort CheatSheet](#) Get started with Snort on CLI  
[pytbull-ng](#)  
[Writing Snort rules](#)  
[Vim Cheat Sheet](#) Most common `vim` editing commands on 1 page

# Lab Instructions

---

**Note:** From here on out, the majority of the work we do in this class will be completed on your VM. **Whenever we are running commands in terminal it should be safe to assume this is done on your VM** unless otherwise specified. For example, this entire lab is to be completed on your virtual machine.



## Exercise 0: Tracking Network Communication

First, we'll make sure all of our packages are up to date by running:

```
sudo apt-get update
```

### Step 0: Download Docker for pytbull-ng

To install docker, run:

```
sudo apt install docker.io
```

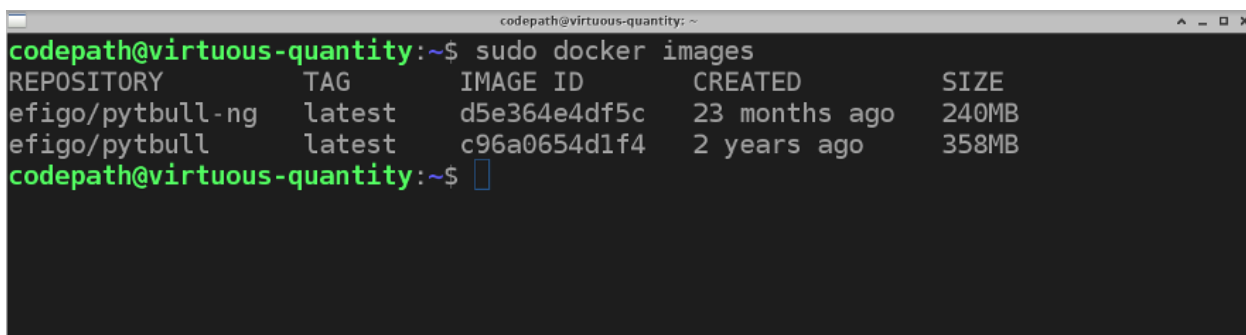
Once docker is installed, run:

```
sudo docker pull efigo/pytbull-ng:latest
```

To ensure that the image has been properly installed use the following command:

```
sudo docker images
```

You should see efigo/pytbull-ng listed.

A terminal window titled 'codepath@virtuous-quantity: ~' showing the output of the 'sudo docker images' command. The output is a table with five columns: REPOSITORY, TAG, IMAGE ID, CREATED, and SIZE. It lists two Docker images: 'efigo/pytbull-ng:latest' with image ID 'd5e364e4df5c' created 23 months ago (240MB) and 'efigo/pytbull:latest' with image ID 'c96a0654d1f4' created 2 years ago (358MB).

| REPOSITORY       | TAG    | IMAGE ID     | CREATED       | SIZE  |
|------------------|--------|--------------|---------------|-------|
| efigo/pytbull-ng | latest | d5e364e4df5c | 23 months ago | 240MB |
| efigo/pytbull    | latest | c96a0654d1f4 | 2 years ago   | 358MB |

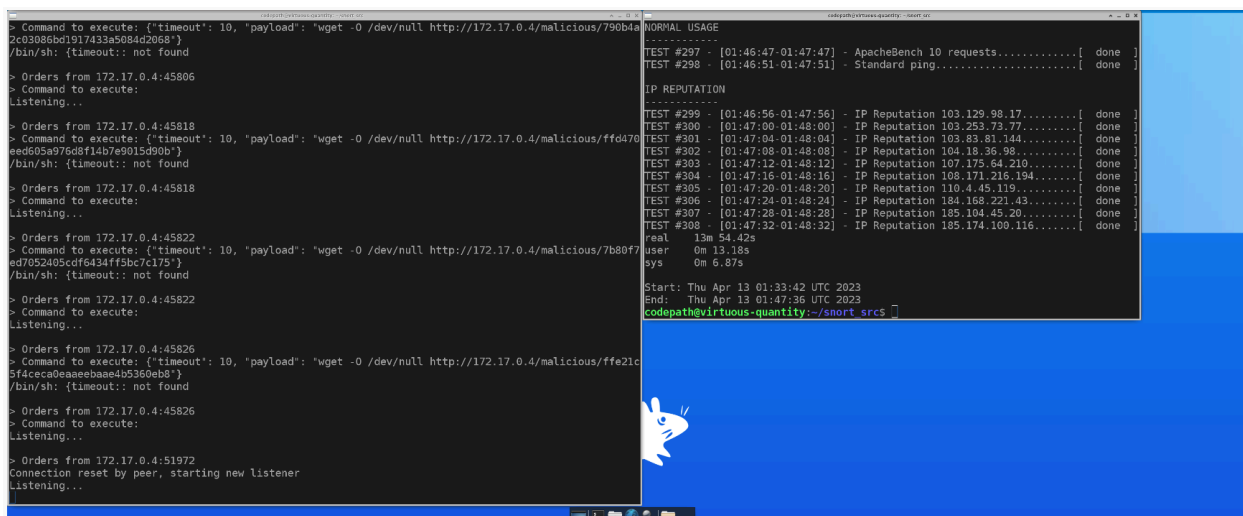
## Step 1: Generating attacks with pytbull-ng

Run each of the the following docker commands in their own terminal windows. (**Tip:** Press `Ctrl + Alt + T` to open a new window):

This command will generate an IP to listen for traffic on (note the IP address after `Host IP:`).

```
sudo docker run --rm -it efigo/pytbull-ng -m victim
```



The image shows two terminal windows. The left window displays a series of commands and responses, including 'Command to execute: {\'timeout\': 10, \'payload\': \'wget -O /dev/null http://172.17.0.4/malicious/790b4a2x03808b01917433a5084e2068\'}', 'Listening...', and 'Orders from 172.17.0.4:45806'. The right window shows 'NORMAL USAGE' with a list of tests (TEST #297 to #308) and their results, including 'ApacheBench 10 requests' and 'Standard ping'. Below the tests, it shows 'IP REPUTATION' with a list of IP addresses and their reputations. At the bottom of the right window, it shows system statistics: 'real 13m 54.42s', 'user 0m 13.18s', 'sys 0m 6.87s', and the start/end times: 'Start: Thu Apr 13 01:33:42 UTC 2023' and 'End: Thu Apr 13 01:47:36 UTC 2023'. The background of the right window is blue with a white mouse cursor icon.

An attacker will start sending packets to our machine, which we can see happening live! Now that we've seen how an attacker is compromising our system, let's learn how to write snort rules to prevent such malicious activity.

## Exercise 1: Writing a Snort Rule to Block Google.com

### Step 0: How to Write Snort Rules

It might be helpful to first read through [this](#) explanation to gain some understanding about how Snort rules are written.

### Step 1: Configuring Snort to Read Rules

We'll be following the [Snort Configuration Guide](#) to set up our system to accept the rules we'll implement in Step 1.

Following these steps without experience editing files and performing other specific tasks can be tricky! So here are a few **tips and clarifications** that should help a bit:

The keyboard shortcut for **paste** in the terminal on your virtual machine is `Ctrl + Shift + V`

In order to edit a few files during these steps (`ethtool.service`, `local.rules`, and `snort.lua`), we'll be using a text editor named **Vim**. Vim is very unique, however, in that we will run it inside the terminal (the command `sudo vi </path/file.name>` opens the specified file in Vim). Vim can do a lot of great things, but can come with its own learning curve. So let's identify just what you need to know to edit these files, and feel free to reference [this](#) guide of Vim commands to learn more.

Vim has two modes, **command** mode and **insert** mode. You will always start in command mode **by default**.

To switch to insert mode, type `i`.

To switch back to command mode, press `Esc`.

If you want to **edit** a file, you will need to be in insert mode. We won't use command mode for now.

So, when you initially run Vim with `sudo vi </path/file.name>`, first hop into insert mode with `i`, make your edits to the file, and then press `Esc`.

Now that we made our changes and are back in command mode, we'll want to **exit** Vim. To do so, we'll type `:wq` and press `Enter`.

`w` writes our changes to memory, and `q` quits Vim.

To get started go ahead and type `sudo vi /lib/systemd/system/ethtool.service` and hit `Enter`. This will create a system script file that we can add to and set to configure every time the system boots up.

Now we'll add the following code to the file:

```
[Unit]
Description=Ethtool Configuration for Network Interface

[Service]
Requires=network.target
Type=oneshot
ExecStart=/sbin/ethtool -K eth0 gro off
ExecStart=/sbin/ethtool -K eth0 lro off

[Install]
WantedBy=multi-user.target
```

Once you're done setting up the file, run `sudo systemctl enable ethtool` to enable the service, and then run `sudo service ethtool start` to start it.

We'll continue by setting up some more folders and files Snort needs, and add a placeholder rule that we will change later.

---

We need to create some folders and files that Snort requires for rules:

```
sudo mkdir /usr/local/etc/rules
sudo mkdir /usr/local/etc/so_rules/
sudo mkdir /usr/local/etc/lists/
sudo touch /usr/local/etc/rules/local.rules
sudo touch /usr/local/etc/lists/default.blocklist
sudo mkdir /var/log/snort
```

We will create one rule in the `local.rules` file that you created above:

```
sudo vi /usr/local/etc/rules/local.rules
```

This rule will detect ICMP traffic, and is really good for testing that Snort is working correctly and generating alerts. Paste the following line into the `local.rules` file (make sure that you're copying this line exactly, you must have a space after each semicolon in this file for `PulledPork` to parse the alert correctly):

*`PulledPork` in an included utility that automates rule fetching and management; we will not be interacting with it directly.*

```
alert icmp any any -> any any ( msg:"ICMP Traffic Detected"; sid:10000001;  
metadata:policy security-ips alert;)
```

Now run Snort and have it load the `local.rules` file (with the `-R` flag) to make sure it loads these rules correctly (verifying the rules are correctly formatted):

(The `-c` flag identifies the location of the snort config file.)

```
snort -c /usr/local/etc/snort/snort.lua -R /usr/local/etc/rules/local.rules
```

The output should end with "Snort successfully validated the configuration". You should not have any warnings or errors. If you scroll up through the output, you should see this rule loaded successfully (under the rule counts section).

```

    stream_file
Finished /usr/local/etc/snort/snort.lua:
Loading rule args:
Loading /usr/local/etc/rules/local.rules:
Finished /usr/local/etc/rules/local.rules:
Finished rule args:
-----
rule counts
    total rules loaded: 1
        text rules: 1
        option chains: 1
        chain headers: 1
-----
port rule counts
    tcp      udp      icmp      ip
any         0        0         1         0
total       0        0         1         0
-----
ips policies rule stats
    id loaded  shared enabled  file
    0      1      0       1  /usr/local/etc/snort/snort.lua
-----
pcap DAQ configured to passive.

Snort successfully validated the configuration (with 0 warnings).
o")~ Snort exiting
codepath@lab000000:~$

```

Now we'll set Snort to detection mode so that it prints out alerts in our terminal. Run the following code **all in the same line** (even though it looks like two separate lines here).

```

sudo snort -c /usr/local/etc/snort/snort.lua -R
/usr/local/etc/rules/local.rules -i eth0 -A alert_fast -s 65535 -k none

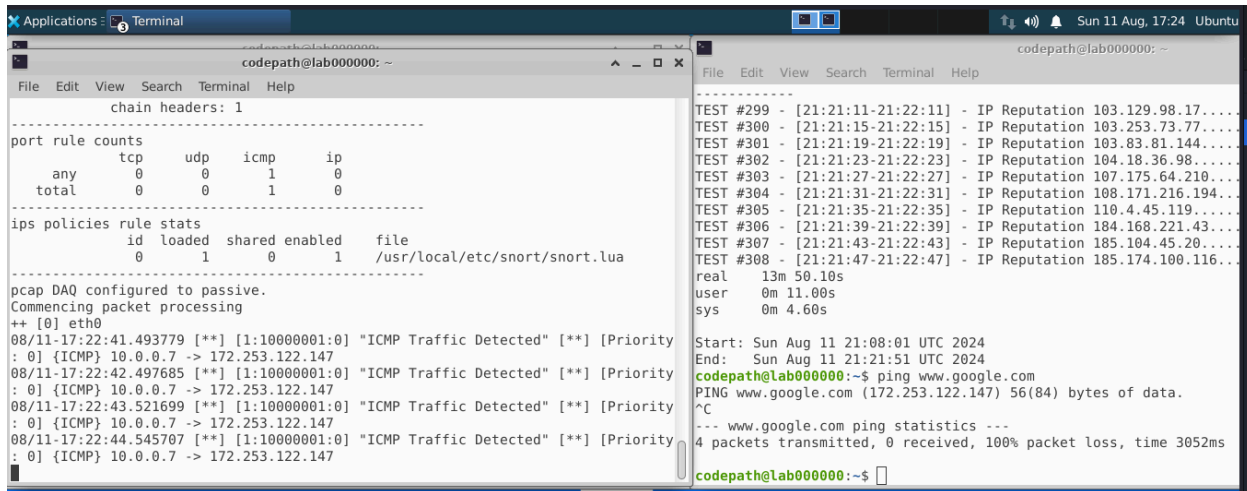
```

Snort should now be listening to all traffic on your virtual machine and checking it with the rule we created. To see what this might look like, we can use the [ping](#) command to test our rule.

Open another terminal window just like we did earlier ([Ctrl + Alt + T](#)), and run [ping google.com](#).

You should see alerts coming in on your first terminal window that the Google IP is being pinged!  
Press `Ctrl + C` to stop pinging.

Let's tie up some loose ends below, and Snort should be all ready to go!



The image shows two terminal windows. The left window displays the Snort configuration output, including chain headers, port rule counts, and IPS policies. The right window shows the output of a ping command to www.google.com, displaying IP reputation and packet statistics.

```
codepath@lab000000: ~
File Edit View Search Terminal Help
-----
chain headers: 1
-----
port rule counts
      tcp      udp      icmp      ip
any      0      0      1      0
total    0      0      1      0
-----
ips policies rule stats
      id loaded shared enabled file
0      1      0      1 /usr/local/etc/snort/snort.lua
-----
pcap DAQ configured to passive.
Commencing packet processing
++ [0] eth0
08/11-17:22:41.493779 [**] [1:10000001:0] "ICMP Traffic Detected" [**] [Priority
: 0] {ICMP} 10.0.0.7 -> 172.253.122.147
08/11-17:22:42.497685 [**] [1:10000001:0] "ICMP Traffic Detected" [**] [Priority
: 0] {ICMP} 10.0.0.7 -> 172.253.122.147
08/11-17:22:43.521699 [**] [1:10000001:0] "ICMP Traffic Detected" [**] [Priority
: 0] {ICMP} 10.0.0.7 -> 172.253.122.147
08/11-17:22:44.545707 [**] [1:10000001:0] "ICMP Traffic Detected" [**] [Priority
: 0] {ICMP} 10.0.0.7 -> 172.253.122.147

codepath@lab000000: ~
File Edit View Search Terminal Help
-----
TEST #299 - [21:21:11-21:22:11] - IP Reputation 103.129.98.17...
TEST #300 - [21:21:15-21:22:15] - IP Reputation 103.253.73.77...
TEST #301 - [21:21:19-21:22:19] - IP Reputation 103.83.81.144...
TEST #302 - [21:21:23-21:22:23] - IP Reputation 104.18.36.98...
TEST #303 - [21:21:27-21:22:27] - IP Reputation 107.175.64.210...
TEST #304 - [21:21:31-21:22:31] - IP Reputation 108.171.216.194...
TEST #305 - [21:21:35-21:22:35] - IP Reputation 110.4.45.119...
TEST #306 - [21:21:39-21:22:39] - IP Reputation 184.168.221.43...
TEST #307 - [21:21:43-21:22:43] - IP Reputation 185.104.45.20...
TEST #308 - [21:21:47-21:22:47] - IP Reputation 185.174.100.116...
real      13m 50.10s
user      0m 11.00s
sys       0m 4.60s
Start: Sun Aug 11 21:08:01 UTC 2024
End: Sun Aug 11 21:21:51 UTC 2024
codepath@lab000000:~$ ping www.google.com
PING www.google.com (172.253.122.147) 56(84) bytes of data.
^C
--- www.google.com ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3052ms
codepath@lab000000:~$
```

Next let's edit the `snort.lua` file. This file is the configuration file we pass to Snort at startup:

```
sudo vi /usr/local/etc/snort/snort.lua
```

Let's enable built-in decoder and inspector alerts and tell the IPS module where our rules file live:

Scroll down to line 170, and look for the section titled **ips**. Here we un-comment (remove the leading two dashes) from `enable_builtin_rules=true`, and enable our `pulledpork` rules.

*Note that lua uses four spaces, not tabs to indent these lines (this is required).*

This section should look like this (comments removed):

```
171 ips =
172 {
173     enable_builtin_rules = true,
174     include = RULE_PATH .. "/local.rules",
175     variables = default_variables
176 }
```

Test your config file (since we've made changes):

```
snort -c /usr/local/etc/snort/snort.lua
```

Now we can run snort as above, however we don't explicitly pass the `local.rules` file on the command line, as we've included it in the **ips** section in the `snort.lua` file:

```
sudo snort -c /usr/local/etc/snort/snort.lua -i eth0 -A alert_fast -s 65535 -k
none
```

## Step 2: Update the Rule to Block HTTPS traffic to Google

Since the current rules we have are just placeholders, we'll want to add new rules that will do exactly what we want -- specifically blocking HTTPS traffic to Google.com.

Use `sudo vi /usr/local/etc/rules/local.rules` to open the rules file, and replace the current rule.

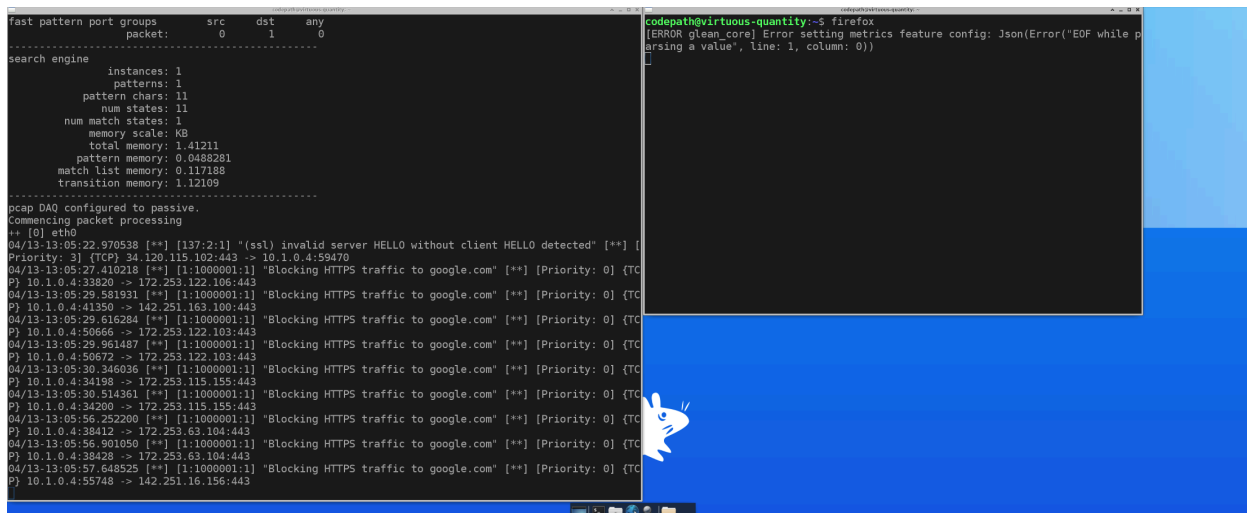
**Hint:** What port does HTTPS use? What protocol?

We can check if we were successful by opening a web browser and going to <https://www.google.com>. Firefox should already be installed on your VM.

**Optional:** In the case that Firefox is **not** pre-installed, use `sudo apt install firefox` to install Firefox. Once that is successful, we can run the browser by simply entering `firefox`.

In another terminal window, run Snort in detection mode again with `sudo snort -c /usr/local/etc/snort/snort.lua -i eth0 -A alert_fast -s 65535 -k none`.

In the Firefox web browser, go to <https://www.google.com>. We should immediately see packets being intercepted because of our Snort rule!



The screenshot shows a terminal window with two panes. The left pane displays Snort logs, including a search engine configuration summary and a list of intercepted packets. The right pane shows a Firefox error message: `[ERROR glean_core] Error setting metrics feature config: Json(Error("EOF while parsing a value", line: 1, column: 0))`. A small white cat icon is visible in the bottom right corner of the terminal window.

```
fast pattern port groups      src  dst  any
packet:      0      1      0
-----
search engine
instances: 1
patterns: 1
pattern chars: 11
num states: 11
num match states: 1
memory scale: KB
total memory: 1.41211
pattern memory: 0.0488281
match list memory: 0.117188
transition memory: 1.12109
-----
pcap DAQ configured to passive.
Commencing packet processing
i: [0] eth0
04/13-13:05:22.970538 [**] [137.2:1] "(ssl) invalid server HELLO without client HELLO detected" [**] [
Priority: 3] [TCP] 34.120.115.102:443 -> 10.1.0.4:59470
04/13-13:05:27.410218 [**] [1:1000000:1] "Blocking HTTPS traffic to google.com" [**] [Priority: 0] [TC
P] 10.1.0.4:33920 -> 172.253.122.106:443
04/13-13:05:29.581931 [**] [1:1000000:1] "Blocking HTTPS traffic to google.com" [**] [Priority: 0] [TC
P] 10.1.0.4:41350 -> 142.251.163.100:443
04/13-13:05:29.616284 [**] [1:1000000:1] "Blocking HTTPS traffic to google.com" [**] [Priority: 0] [TC
P] 10.1.0.4:50666 -> 172.253.122.103:443
04/13-13:05:29.961487 [**] [1:1000000:1] "Blocking HTTPS traffic to google.com" [**] [Priority: 0] [TC
P] 10.1.0.4:50672 -> 172.253.122.103:443
04/13-13:05:30.346036 [**] [1:1000000:1] "Blocking HTTPS traffic to google.com" [**] [Priority: 0] [TC
P] 10.1.0.4:34198 -> 172.253.115.155:443
04/13-13:05:30.514361 [**] [1:1000000:1] "Blocking HTTPS traffic to google.com" [**] [Priority: 0] [TC
P] 10.1.0.4:34200 -> 172.253.115.155:443
04/13-13:05:56.252200 [**] [1:1000000:1] "Blocking HTTPS traffic to google.com" [**] [Priority: 0] [TC
P] 10.1.0.4:38412 -> 172.253.63.104:443
04/13-13:05:56.901050 [**] [1:1000000:1] "Blocking HTTPS traffic to google.com" [**] [Priority: 0] [TC
P] 10.1.0.4:38428 -> 172.253.63.104:443
04/13-13:05:57.648525 [**] [1:1000000:1] "Blocking HTTPS traffic to google.com" [**] [Priority: 0] [TC
P] 10.1.0.4:55748 -> 142.251.16.156:443
```

You may notice that we can still technically access Google.com, and so the last step, if we truly wanted to block traffic, would be to drop the packets instead of alerting when we receive them. But that can be for another day.

🎉 Congratulations, you've completed the lab! 🎉